



Apple Store Sales SQL Data Analysis

Real Data | Powerful Insights | Better Decisions



PostgreSQL

1M+ Records

21 Business Questions



PostgreSQL



SQL



GitHub



Data Analysis

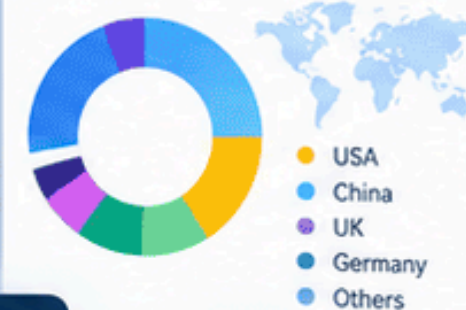
Sales Trend



Top Products



Sales by Country



```
SELECT
  p.product_name,
  SUM(s.quantity) AS total_sold
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.product_name
ORDER BY total_sold DESC;
```



5 Tables
(1M+ Rows)



Window Functions
(RANK, LAG)



CTEs
(Common Table Expressions)

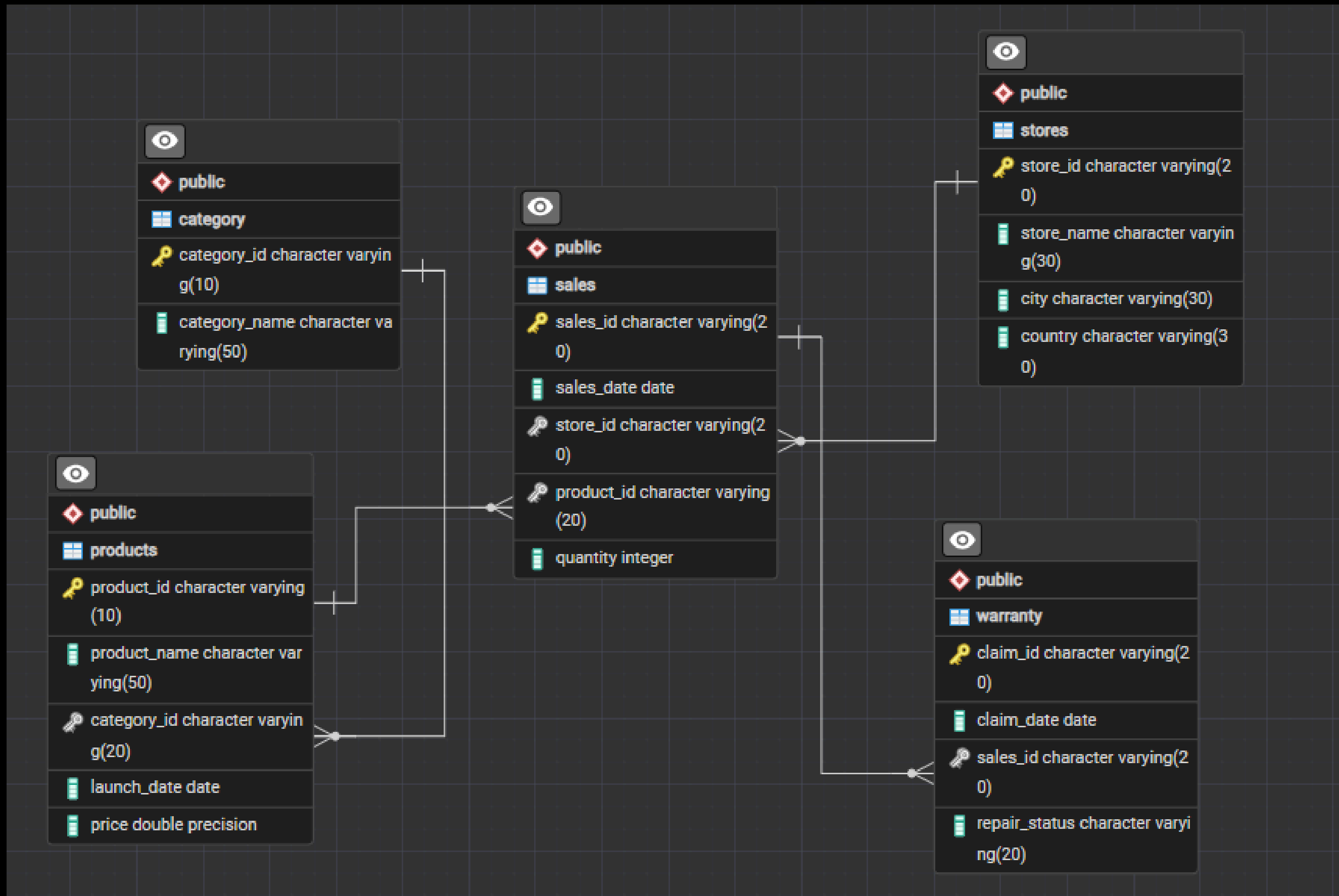


Advanced Joins
(INNER, LEFT, RIGHT)



Business Insights
(Real-World Analysis)

EDR



48

49

50

51

52

53

54

55

56

57

Data Output

Messages

Notifications










 SQL

Showing rows: 1 to 75

Page No:

1

of 1

	store_id character varying (20)	store_name character varying (30)	total_units bigint
1	ST-56	Apple Southland	77795
2	ST-34	Apple Fukuoka	77787
3	ST-1	Apple Fifth Avenue	77689
4	ST-30	Apple Dubai Mall	77571
5	ST-24	Apple Kurfuerstendamm	77532
6	ST-39	Apple Taipei 101	77518
7	ST-75	Apple Beijing SKP	77482
8	ST-25	Apple Schildergasse	77385
9	ST-5	Apple SoHo	77186
10	ST-52	Apple Chadstone	77170
11	ST-12	Apple North Michigan Aven...	77152

Total rows: 75

Query complete 00:00:00.847

CRLF



Query Query History

```
59 -- 3. Identify how many sales occurred in December 2023.  
60 SELECT COUNT(*)  
61 FROM sales  
62 WHERE TO_CHAR(sales_date, 'MM-YYYY') = '12-2023';  
63
```

Data Output Messages Notifications



Showing rows: 1 to 1   Page No.

	count bigint 
1	18076

Query Query History

```
64  -- 4. Determine how many stores have never had a warranty claim filed.
65  SELECT COUNT(*) FROM stores AS st
66  WHERE st.store_id NOT IN (
67          SELECT DISTINCT store_id FROM sales AS sl
68          RIGHT JOIN warranty AS w
69          ON sl.sales_id = w.sales_id
70      );
71
```

Data Output Messages Notifications



Showing rows: 1 to 1   Page No:

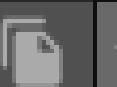
count
bigint 

1	0
---	---

Query Query History

```
71
72 -- 5. Calculate the percentage of warranty claims marked as "Warranty Rejected".
73 SELECT
74     ROUND(COUNT(*) /
75     (SELECT COUNT(*) FROM warranty) :: numeric
76     *100,2) AS warranty_rejected_percentage
77 FROM warranty
78 WHERE repair_status = 'Rejected';
79
80
```

Data Output Messages Notifications



SQL

Showing rows: 1 to 1   Page N

	warranty_rejected_percentage numeric 
1	24.52

```
81 -- 6. Identify which store had the highest total units sold in the last 2 year.
82 WITH abc AS
83     (SELECT store_id, SUM(quantity) AS total_unit
84     FROM sales
85     WHERE sales_date >= (SELECT CURRENT_DATE - INTERVAL '2 year')
86     GROUP BY store_id)
87 SELECT
88     s.store_id, s.store_name, abc.total_unit
89 FROM stores AS s
90 LEFT JOIN abc
91 ON s.store_id = abc.store_id
92 ORDER BY abc.total_unit DESC
93 LIMIT 1;
94
```

≡+

📄

▼

📋

▼

🗑

🗑

📥

📥

📄

SQL

Showing rows: 1 to 1

✎

📄

 Page No:

1

 of 1

⏪

	store_id [PK] character varying (20) ✎	store_name character varying (30) ✎	total_unit bigint 🔒
1	ST-2	Apple Union Square	3337

Query Query History

```

99
100 -- 7. Count the number of unique products sold in the last 2 year.
101 -- Each Product sold.
102
103 SELECT
104     product_id,
105     COUNT(*) AS sold
106 FROM sales
107 WHERE sales_date >= (SELECT CURRENT_DATE - INTERVAL '2 year')
108 GROUP BY 1
109 ORDER BY 2 DESC;
110

```

Data Output Messages Notifications

≡+

📄

▼

📋

▼

🗑️

🗃️

⬇️

📈

SQL

Showing rows: 1 to 89   Page No: 1

	product_id character varying (20) 🔒	sold bigint 🔒
1	P-29	529
2	P-23	508
3	P-56	506
4	P-12	500
5	P-57	497
6	P-53	496
7	P-88	494
8	P-84	490
9	P-52	489
10	P-67	487

Total rows: 89

Query complete 00:00:00.395

Query

Query History

116

117

118

119

120

121

122

123

124

125

126

127

-- 8. Find the average price of products in each category.
SELECT
 p.category_id,
 c.category_name,
 AVG(p.price) AS avg_price
FROM products AS p
JOIN category AS c
ON c.category_id = p.category_id
GROUP BY 1,2
ORDER BY avg_price **DESC**;
|

Data Output

Messages

Notifications

≡+

SQL

Showing rows: 1 to 10

Page No:

	category_id character varying (20)	category_name character varying (50)	avg_price double precision
1	CAT-3	Tablet	1479.5
2	CAT-1	Laptop	1194.1
3	CAT-5	Wearable	1146.8888888888889
4	CAT-2	Audio	1124.8181818181818
5	CAT-4	Smartphone	1037.5384615384614
6	CAT-10	Accessories	1030.142857142857
7	CAT-6	Streaming Device	996.6666666666666
8	CAT-7	Desktop	838.7
9	CAT-8	Subscription Service	823.2857142857143
10	CAT-9	Smart Speaker	734

Total rows: 10

Query complete 00:00:00.176

Query

Query History

```
127
128
129 -- 9. How many warranty claims were filed Completed?
130
131 SELECT
132     COUNT(*)
133     -- EXTRACT(YEAR FROM claim_date) AS claim_year      -- Learn how to extract year from any date
134 FROM warranty
135 WHERE repair_status = 'Completed';
136
```

Data Output

Messages

Notifications



Showing rows: 1 to 1



Page No:

1

	count bigint 
1	7466

Query Query History

```
137 -- 10. For each store, identify the best-selling day based on highest quantity sold.
138
139 SELECT * FROM(
140     SELECT
141         sl.store_id,
142         TO_CHAR(sl.sales_date, 'Day') AS sale_day,
143         SUM((p.price * sl.quantity)) AS net_price,
144         RANK() OVER (PARTITION BY sl.store_id ORDER BY SUM((p.price * sl.quantity)) DESC) AS rank
145     FROM sales AS sl
146     LEFT JOIN products AS p
147     ON sl.product_id = p.product_id
148     GROUP BY 1,2) AS tb1
149 WHERE rank = 1;
```

Data Output Messages Notifications












Showing rows: 1 to 75 

Page No: 1

of 1

	store_id character varying (20) 🔒	sale_day text 🔒	net_price double precision 🔒	rank bigint 🔒
1	ST-1	Tuesday	12168968	1
2	ST-10	Tuesday	12109477	1
3	ST-11	Thursday	12769991	1
4	ST-12	Thursday	12088152	1
5	ST-13	Sunday	12418839	1
6	ST-14	Monday	12109068	1
7	ST-15	Thursday	11794485	1
8	ST-16	Tuesday	12067295	1

Total rows: 75

Query complete 00:00:03.281

CRLF



Query Query History

```
162 -- 11. Identify the least selling product in each country for each year based on total units sold.
163
164 WITH tb1 AS (
165     SELECT
166         st.country,
167         p.product_name,
168         SUM(sl.quantity) AS total_unit,
169         RANK() OVER(PARTITION BY st.country ORDER BY SUM(sl.quantity)) AS rank
170     FROM sales AS sl
171     JOIN stores AS st
172     ON sl.store_id = st.store_id
173     JOIN products AS p
174     ON p.product_id = sl.product_id
175     GROUP BY 1,2
176 )
177 SELECT * FROM tb1
178 WHERE rank = 1;
```

Data Output Messages Notifications

≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL

Showing rows: 1 to 19 ✎ 📄 | Page No: 1 of

	country character varying (30) 🔒	product_name character varying (50) 🔒	total_unit bigint 🔒	rank bigint 🔒
1	Australia	MacBook Air (M1)	5516	1
2	Austria	Apple Music	663	1
3	Canada	Apple Pencil (1st Generati...	3798	1
4	China	iPhone 12	5343	1
5	Colombia	Beats Studio Buds	1428	1

Total rows: 19

Query complete 00:00:01.182

Query Query History

```
179
180 -- 12. Calculate how many warranty claims were filed within 180 days of a product sale.
181
182 WITH abc AS (
183     SELECT
184         sl.sales_id,
185         sl.sales_date,
186         sl.product_id,
187         w.claim_date,
188         (w.claim_date - sl.sales_date) AS diff_date
189     FROM sales AS sl
190     JOIN warranty AS w
191     ON sl.sales_id = w.sales_id
192     WHERE (w.claim_date - sl.sales_date) <= 180)
193 SELECT COUNT(*) FROM abc
194 WHERE diff_date >= 0;
195
```

Data Output Messages Notifications



Showing rows: 1 to 1 Pa

	count bigint
1	3046

Query Query History

```
196 -- 13. Determine how many warranty claims were filed for products launched in the last two years.
197
198 SELECT
199     COUNT(*)
200     -- sl.sales_id,
201     -- sl.product_id,
202     -- p.launch_date,
203     -- w.claim_date,
204     -- (claim_date - launch_date) AS diff_date
205 FROM sales AS sl
206 JOIN products AS p
207 ON sl.product_id = p.product_id
208 JOIN warranty AS w
209 ON w.sales_id = sl.sales_id
210 WHERE ((w.claim_date - p.launch_date) <= 730)
211        AND ((w.claim_date - p.launch_date) >= 0);
```

Data Output Messages Notifications



Showing rows: 1 to 1



Page No: 1

	count bigint
1	11369

Query

Query History

214

-- 14. List the months in the last three years where sales exceeded 5,000 units in the USA.

215

SELECT

216

st.country,

217

TO_CHAR(sl.sales_date, 'MM-YYYY'),

218

SUM(sl.quantity) AS total_unit

219

FROM sales AS sl

220

JOIN stores AS st

221

ON st.store_id = sl.store_id

222

WHERE

223

(country = 'United States')

224

AND

225

(sl.sales_date >= CURRENT_DATE - INTERVAL '3 YEAR')

226

GROUP BY 1,2

227

HAVING SUM(sl.quantity) > 5000;

228

≡+

📄

▼

📋

▼

🗑

🗑

📥

📥

📄

SQL

Showing rows: 1 to 15

✎

🖨

 | Page No: 1

	country character varying (30) 🔒	to_char text 🔒	total_unit bigint 🔒
1	United States	04-2024	19513
2	United States	10-2023	20238
3	United States	11-2024	7956
4	United States	09-2024	19203
5	United States	03-2024	20051
6	United States	11-2023	18926
7	United States	10-2024	19775
8	United States	04-2024	19513
9	United States	03-2024	20051
10	United States	09-2024	19203
11	United States	11-2024	7956
12	United States	10-2024	19775
13	United States	04-2024	19513
14	United States	03-2024	20051
15	United States	10-2024	19775

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

```
-- 15. Identify the product category with the most warranty claims filed in the last two years.

SELECT
    c.category_name,
    COUNT(w.claim_id) AS total_claim
FROM warranty AS w
LEFT JOIN sales AS sl
ON w.sales_id = sl.sales_id
JOIN products AS p
ON p.product_id = sl.product_id
JOIN category AS c
ON c.category_id = p.category_id
WHERE w.claim_date >= CURRENT_DATE -INTERVAL '2 YEAR'
GROUP BY 1
ORDER BY total_claim DESC
LIMIT 1;
```



SQL

	category_name character varying (50) 	total_claim bigint 
1	Accessories	1026

Query Query History

```
254
255 -- 16. Determine the percentage chance of receiving warranty claims after each purchase for each country.
256 SELECT
257     *,
258     ((total_claim :: numeric / total_unit :: numeric) * 100) AS risk
259 FROM
260     (SELECT
261         st.country,
262         SUM(sl.quantity) AS total_unit,
263         COUNT(w.claim_id) AS total_claim
264     FROM sales AS sl
265     JOIN stores AS st
266     ON sl.store_id = st.store_id
267     LEFT JOIN warranty AS w
268     ON w.sales_id = sl.sales_id
269     GROUP BY 1) AS tbl
270 ORDER BY 4 DESC;
```

Data Output Messages Notifications

         SQL

Showing rows: 1 to 19



Page No:

1

	country character varying (30) 🔒	total_unit bigint 🔒	total_claim bigint 🔒	risk numeric 🔒
1	Austria	75965	448	0.58974527743039557700
2	Netherlands	76764	428	0.55755301964462508500
3	Taiwan	77518	419	0.54051962124925823700
4	UAE	381972	2046	0.53564135591090446400
5	Canada	381212	2035	0.53382369914902993600

Total rows: 19

Query complete 00:00:00.913

```
-- 17. Analyze the year-by-year growth ratio for each store.
WITH tb1 AS
    (SELECT
        st.store_name,
        EXTRACT(YEAR FROM sl.sales_date) AS sales_year,
        SUM(sl.quantity * p.price ) AS current_sales
    FROM sales AS sl
    JOIN products AS p
    ON p.product_id = sl.product_id
    JOIN stores AS st
    ON st.store_id = sl.store_id
    GROUP BY 1,2),
tb2 AS
    (SELECT
        tb1.*,
        LAG(current_sales, 1) OVER(PARTITION BY store_name ORDER BY sales_year) AS previous_sales
    FROM tb1)
SELECT
    tb2.*,
    ROUND(((tb2.current_sales - tb2.previous_sales) :: numeric / tb2.previous_sales :: numeric) * 100,2) AS growth
FROM tb2
WHERE (previous_sales IS NOT NULL)
AND
    (sales_year <> 2024);      -- Current year(2024) is running that's we are ignoring it.
```

Data OutputMessagesNotifications

	store_name character varying (30)	sales_year numeric	current_sales double precision	previous_sales double precision	growth numeric
1	Apple Ala Moana	2021	16351838	16736679	-2.30
2	Apple Ala Moana	2022	16679728	16351838	2.01
3	Apple Ala Moana	2023	17286752	16679728	3.64
4	Apple Andino	2021	16657275	15873867	4.94
5	Apple Andino	2022	16807255	16657275	0.90
6	Apple Andino	2023	16452478	16807255	-2.11
7	Apple Antara	2021	16516324	16614977	-0.59
8	Apple Antara	2022	16625607	16516324	0.66
9	Apple Antara	2023	15789600	16625607	-5.03
10	Apple Beijing SKP	2021	16967537	17004025	-0.21
11	Apple Beijing SKP	2022	17293828	16967537	1.92
12	Apple Beijing SKP	2023	16793173	17293828	-2.89
13	Apple Beverly Center	2021	16086769	16876768	-4.68
14	Apple Beverly Center	2022	16653180	16086769	3.52
15	Apple Beverly Center	2023	16945007	16653180	1.75
16	Apple Bondi	2021	16883063	16887187	-0.02
Total rows: 207		Query complete 00:00:03.317			

Query Query History

```
305 -- 18. Calculate the correlation between product price and warranty claims
306 -- for products sold in the last five years, segmented by price range.
307 SELECT
308     CASE
309         WHEN p.price < 500 THEN 'Less Expensive'
310         WHEN p.price BETWEEN 500 AND 1000 THEN 'Mid Range Expensive'
311         ELSE 'Very Expensive Product'
312     END,
313     COUNT(w.claim_id) AS claimed_item
314 FROM warranty AS w
315 LEFT JOIN sales AS sl
316 ON sl.sales_id = w.sales_id
317 JOIN products AS p
318 ON p.product_id = sl.product_id
319 WHERE sl.sales_date >= CURRENT_DATE - INTERVAL '5 YEAR'
320 GROUP BY 1;
```

Data Output Messages Notifications



Showing rows: 1 to 3



Page No: 1

	case text	claimed_item bigint
1	Less Expensive	3321
2	Mid Range Expensive	5355
3	Very Expensive Prod...	11021

Query History

```
-- 19. Identify the store with the highest percentage of "Completed" claims relative to total claims filed.
WITH tb1 AS
    (SELECT
        st.store_id, COUNT(w.claim_id) AS Completed_Claim
    FROM warranty AS w
    JOIN sales AS sl ON w.sales_id = sl.sales_id
    JOIN stores AS st ON st.store_id = sl.store_id
    WHERE w.repair_status = 'Completed'
    GROUP BY 1),
tb2 AS
    (SELECT
        st.store_id, COUNT(w.claim_id) AS All_Claim
    FROM warranty AS w
    JOIN sales AS sl ON w.sales_id = sl.sales_id
    JOIN stores AS st ON st.store_id = sl.store_id
    GROUP BY 1)
SELECT
    tb1.store_id, st.store_name, tb1.completed_claim, tb2.all_claim,
    ROUND((tb1.completed_claim :: numeric / tb2.all_claim :: numeric) * 100, 2) AS completed_claim_percentage
FROM tb1 JOIN tb2
ON tb1.store_id = tb2.store_id
JOIN stores AS st
ON st.store_id = tb1.store_id
ORDER BY 5 DESC;
```

Data Output		Messages
	store_id [PK] character varying	
1	ST-65	

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

📁

📶

📈

SQL

Showing row

	store_id [PK] character varying (20)	store_name character varying (30)	completed_claim bigint	all_claim bigint	completed_claim_percentage numeric
1	ST-65	Apple Iconsiam	114	377	30.24
2	ST-59	Apple Mall of the Emirates	110	365	30.14
3	ST-16	Apple Regent Street	127	437	29.06
4	ST-55	Apple Highpoint	118	413	28.57
5	ST-40	Apple Causeway Bay	105	368	28.53
6	ST-8	Apple Pioneer Place	108	381	28.35
7	ST-62	Apple Via Santa Fe	116	414	28.02
8	ST-48	Apple Sainte-Catherine	108	386	27.98
9	ST-69	Apple The Dubai Mall	116	419	27.68
10	ST-33	Apple Marunouchi	109	395	27.59
11	ST-25	Apple Schildergasse	111	404	27.48
12	ST-57	Apple Yas Mall	110	405	27.16
13	ST-31	Apple Shinjuku	120	442	27.15
14	ST-71	Apple Yeouido	102	376	27.13
15	ST-35	Apple Shanghai IFC	103	381	27.03

Total rows: 75

Query complete 00:00:00.570

Query Query History

```

358 -- 20. Write a query to calculate the monthly running total of sales for each store
359 -- over the past four years and compare trends during this period.
360 WITH tb1 AS
361     (SELECT
362         sl.store_id,
363         EXTRACT(YEAR FROM sl.sales_date) AS sales_year,
364         EXTRACT(MONTH FROM sl.sales_date) AS sales_month,
365         SUM(sl.quantity * p.price ) AS total_revenue
366     FROM sales AS sl
367     LEFT JOIN products AS p ON sl.product_id = p.product_id
368     GROUP BY 1,2,3
369     ORDER BY 1,2,3)
370 SELECT
371     store_id, sales_year, sales_month, total_revenue,
372     SUM(total_revenue) OVER(PARTITION BY store_id ORDER BY sales_year, sales_month) AS running_total
373 FROM tb1;

```

Data Output Messages Notifications

≡+

📄

▼

📋

▼

🗑️

🏠

⬇️

📈

SQL

Showing rows: 1 to 1000

✎

🖨️

 Page No: 1

	store_id character varying (20) 🔒	sales_year numeric 🔒	sales_month numeric 🔒	total_revenue double precision 🔒	running_total double precision 🔒
1	ST-1	2020	1	1519710	1519710
2	ST-1	2020	2	1513077	3032787
3	ST-1	2020	3	1456142	4488929
4	ST-1	2020	4	1432050	5920979
5	ST-1	2020	5	1408946	7329925
6	ST-1	2020	6	1398598	8728523

Total rows: 4425

Query complete 00:00:04.039


```
377 -- 21. Analyze product sales trends over time, segmented into key periods:
378 -- from launch to 6 months, 6-12 months, 12-18 months, and beyond 18 months.
```

```
380 WITH tab1 AS
381     (SELECT
382         p.product_name,
383         CASE
384             WHEN sl.sales_date BETWEEN p.launch_date AND p.launch_date + INTERVAL '6 MONTH' THEN '0-6'
385             WHEN sl.sales_date BETWEEN p.launch_date + INTERVAL '6 MONTH' AND p.launch_date + INTERVAL '12 MONTH' THEN '6-12'
386             WHEN sl.sales_date BETWEEN p.launch_date + INTERVAL '12 MONTH' AND p.launch_date + INTERVAL '18 MONTH' THEN '12-18'
387             ELSE '18+'
388         END AS plc,
389         SUM(sl.quantity) AS total_qty_sale
390     FROM sales AS sl
391     JOIN products AS p
392     ON p.product_id = sl.product_id
393     GROUP BY 1,2)
394 SELECT * FROM tab1
395 ORDER BY 1,
396     CASE
397         WHEN plc = '0-6' THEN 1
398         WHEN plc = '6-12' THEN 2
399         WHEN plc = '12-18' THEN 3
400         ELSE 4
401     END;
402
```

Data OutputMessagesNotifications

SQL

	product_name character varying (50)	plc text	total_qty_sale bigint
1	AirPods (2nd Generation)	0-6	6650
2	AirPods (2nd Generation)	6-12	6635
3	AirPods (2nd Generation)	12-18	6615
4	AirPods (2nd Generation)	18+	44681
5	AirPods (3rd Generation)	0-6	6536
6	AirPods (3rd Generation)	6-12	6652
7	AirPods (3rd Generation)	12-18	6878
8	AirPods (3rd Generation)	18+	44036
9	AirPods Max	0-6	6606
10	AirPods Max	6-12	6881
11	AirPods Max	12-18	6511
12	AirPods Max	18+	44846
13	AirPods Pro	0-6	1394
14	AirPods Pro	18+	63349
15	AirPods Pro (2nd Generatio...	0-6	6788
16	AirPods Pro (2nd Generatio...	6-12	6383
17	AirPods Pro (2nd Generatio...	12-18	6612
Total rows: 326		Query complete 00:00:03.019	